Algoritmos - Actividad Guiada 1

Nombre: Marc González Pastor
URL: https://colab.research.google.com/github/marcgpastor/03_MIAR_Actividades/blob/master/02_Algoritmos_AG1.ipynb
Repositorio GitHub: https://github.com/marcgpastor/03_MIAR_Actividades

Torres de Hanoi con Divide y vencerás

```
def Torres_Hanoi(N, desde, hasta):
  if N == 1:
    print("Lleva el disco", 1, "desde", desde, "hasta", hasta)
  else:
    aux = 6 - desde - hasta  # poste auxiliar (1+2+3=6)
    Torres_Hanoi(N-1, desde, aux)
    print("Lleva el disco", N, "desde", desde, "hasta", hasta)
    Torres_Hanoi(N-1, aux, hasta)

Torres_Hanoi(3, 1, 3)
```

```
Lleva el disco 1 desde 1 hasta 3
Lleva el disco 2 desde 1 hasta 2
Lleva el disco 1 desde 3 hasta 2
Lleva el disco 3 desde 1 hasta 3
Lleva el disco 1 desde 2 hasta 1
Lleva el disco 2 desde 2 hasta 3
Lleva el disco 1 desde 1 hasta 3
```

```
def Fibonacci(N: int):
    if N < 0:
        raise ValueError("N debe ser >= 0")
    if N == 0:
        return 0
    if N == 1:
        return 1
    return Fibonacci(N-1) + Fibonacci(N-2)

Fibonacci(6)
```

```
8
```

Devolución de cambio por técnica voraz

```
def cambio_monedas(N, SM):
    SM = sorted(SM, reverse=True)

    SOLUCION = [0] * len(SM)
    ValorAcumulado = 0

    for i, valor in enumerate(SM):
        monedas = (N - ValorAcumulado) // valor
        SOLUCION[i] = monedas
        ValorAcumulado += monedas * valor

    return SOLUCION if ValorAcumulado == N else None

cambio_monedas(15,[25,10,5,1])
```

```
[0, 1, 1, 0]
```

N-Reinas por técnica de vueta atrás

```
def escribe(S):
  n = len(S)
  for x in range(n):
    print("")
    for i in range(n):
      if S[i] == x+1:
        print(" X " , end="")
      else:
        print(" - ", end="")

def es_prometedora(SOLUCION, etapa):
    # Misma fila que alguna reina anterior
    if SOLUCION[etapa] in SOLUCION[:etapa]:
        return False

    # Diagonales con reinas anteriores
```

```
    for j in range(etapa):
        if abs(etapa - j) == abs(SOLUCION[etapa] - SOLUCION[j]):
            return False

    return True

def reinas(N, solucion=None, etapa=0):
    if solucion is None:
        solucion = [0 for _ in range(N)]

    for fila in range(1, N + 1):
        solucion[etapa] = fila

        if es_prometedora(solucion, etapa):
            if etapa == N - 1:
                print(solucion)
                print()
            else:
                reinas(N, solucion, etapa + 1)

        solucion[etapa] = 0

reinas(8)
```

```
[1, 5, 8, 6, 3, 7, 2, 4]

[1, 6, 8, 3, 7, 4, 2, 5]

[1, 7, 4, 6, 8, 2, 5, 3]

[1, 7, 5, 8, 2, 4, 6, 3]

[2, 4, 6, 8, 3, 1, 7, 5]

[2, 5, 7, 1, 3, 8, 6, 4]

[2, 5, 7, 4, 1, 8, 6, 3]

[2, 6, 1, 7, 4, 8, 3, 5]

[2, 6, 8, 3, 1, 4, 7, 5]

[2, 7, 3, 6, 8, 5, 1, 4]

[2, 7, 5, 8, 1, 4, 6, 3]

[2, 8, 6, 1, 3, 5, 7, 4]

[3, 1, 7, 5, 8, 2, 4, 6]

[3, 5, 2, 8, 1, 7, 4, 6]

[3, 5, 2, 8, 6, 4, 7, 1]

[3, 5, 7, 1, 4, 2, 8, 6]

[3, 5, 8, 4, 1, 7, 2, 6]

[3, 6, 2, 5, 8, 1, 7, 4]

[3, 6, 2, 7, 1, 4, 8, 5]

[3, 6, 2, 7, 5, 1, 8, 4]

[3, 6, 4, 1, 8, 5, 7, 2]

[3, 6, 4, 2, 8, 5, 7, 1]

[3, 6, 8, 1, 4, 7, 5, 2]

[3, 6, 8, 1, 5, 7, 2, 4]

[3, 6, 8, 2, 4, 1, 7, 5]

[3, 7, 2, 8, 5, 1, 4, 6]

[3, 7, 2, 8, 6, 4, 1, 5]

[3, 8, 4, 7, 1, 6, 2, 5]

[4, 1, 5, 8, 2, 7, 3, 6]
```

Viaje por el rio. Programación dinámica

```
INF = float("inf")

tarifas = [
    [0, 5, 4, 3, INF, INF, INF],
    [INF, 0, INF, 2, 3, INF, 11],
    [INF, INF, 0, 1, INF, 4, 10],
```

```
    [INF, INF, INF, 0, 5, 6, 9],
    [INF, INF, INF, INF, 0, INF, 4],
    [INF, INF, INF, INF, INF, 0, 3],
    [INF, INF, INF, INF, INF, INF, 0]
]
```

```
from typing import List, Tuple, Optional

def calcular_precios_ruta(tarifas: List[List[float]]) -> Tuple[List[List[float]], List[List[Optional[int]]]]:
    """Calcula las matrices de precios mínimos y nodos para reconstruir rutas.

    precios[i][j] almacena el coste mínimo para ir del nodo i al nodo j (solo para i < j).
    ruta[i][j] almacena el nodo k que permite obtener ese coste mínimo en el último tramo k -> j.
    Si la mejor opción es ir directamente de i a j, entonces ruta[i][j] = i.

    Nota: este algoritmo rellena únicamente la parte superior de las matrices (i < j), asumiendo
    que el recorrido avanza hacia adelante (no hay ciclos ni retrocesos).

    Args:
        tarifas: Matriz cuadrada con los costes directos. Si no hay un camino directo, se debe usar
            un valor alto (ej. float('inf')).

    Returns:
        Una tupla (precios, ruta) donde:
        - precios: Matriz con los costes mínimos para ir de i a j.
        - ruta: Matriz para reconstruir la ruta óptima. Las posiciones no calculadas (i >= j) se dejan como None.
    """
    # Total de nodos
    n = len(tarifas)        # Al ser una matriz cuadrada, len(tarifas) == len(tarifas[0])

    # Inicialización de las matrices de precios y rutas
    precios: List[List[float]] = [[INF] * n for _ in range(n)]
    ruta: List[List[Optional[int]]] = [[None] * n for _ in range(n)]
    for i in range(n):
        precios[i][i] = 0.0
        ruta[i][i] = i

    # Construcción de precios y ruta para cada par (origen, destino)
    for i in range(n - 1):              # Origen
        for j in range(i + 1, n):       # Destino
            minimo = tarifas[i][j]
            ruta[i][j] = i              # Inicialmente, asumimos que la mejor ruta es la directa

            for k in range(i + 1, j):   # Nodo intermedio
                candidato = precios[i][k] + tarifas[k][j]
                if candidato < minimo:
                    minimo = candidato
                    ruta[i][j] = k      # Actualizamos el nodo intermedio

            precios[i][j] = minimo      # Guardamos el coste mínimo encontrado

    return precios, ruta
```

```
def calcular_ruta(ruta: List[List[Optional[int]]], origen: int, destino: int) -> str:
    """Reconstruye la ruta óptima a partir de la matriz `ruta`.

    La matriz `ruta` se interpreta así:
      - ruta[i][j] = i si el mejor camino es directo i -> j.
      - ruta[i][j] = k (con i < k < j) si el mejor camino termina con k -> j
      y el tramo i -> k debe reconstruirse recursivamente.

    Args:
        ruta: Matriz devuelta por `calcular_precios_ruta`.
        origen: Nodo de inicio.
        destino: Nodo de destino.

    Returns:
        Ruta completa como cadena con nodos separados por comas (p. ej. "0,2,4,6").
    """
    k = ruta[origen][destino]
    if k is None:
        raise ValueError(f"No hay información de ruta para ({origen}, {destino}).")

    if k == origen:
        return f"{origen},{destino}"

    return f"{calcular_ruta(ruta, origen, k)},{destino}"
```

```
precios, ruta = calcular_precios_ruta(tarifas)

origen = 0
destino = 6
print("Coste mínimo 0→6:", precios[origen][destino])
print("Ruta 0→6:", calcular_ruta(ruta, origen, destino))

print("\nPRECIOS")
```

```
for fila in precios:
    print(fila)

print("\nRUTA")
for fila in ruta:
    print(fila)
```

```
Coste mínimo 0→6: 11
Ruta 0→6: 0,2,5,6

PRECIOS
[0.0, 5, 4, 3, 8, 8, 11]
[inf, 0.0, inf, 2, 3, 8, 7]
[inf, inf, 0.0, 1, 6, 4, 7]
[inf, inf, inf, 0.0, 5, 6, 9]
[inf, inf, inf, inf, 0.0, inf, 4]
[inf, inf, inf, inf, inf, 0.0, 3]
[inf, inf, inf, inf, inf, inf, 0.0]

RUTA
[0, 0, 0, 0, 1, 2, 5]
[None, 1, 1, 1, 1, 3, 4]
[None, None, 2, 2, 3, 2, 5]
[None, None, None, 3, 3, 3, 3]
[None, None, None, None, 4, 4, 4]
[None, None, None, None, None, 5, 5]
[None, None, None, None, None, None, 6]
```

APORTACIÓN EXTRA: Dos puntos más cercanos

Problema: dado un conjunto de puntos, encontrar los dos más cercanos (menor distancia).

1. Generar datos aleatorios en **1D**.
2. **Primer intento (Fuerza Bruta)** + calcular complejidad. ¿Se puede mejorar?
3. **Segundo intento (Divide y Vencerás)** en 1D + calcular complejidad. ¿Se puede mejorar?
4. Extender el algoritmo a **2D**.
5. Extender el algoritmo a **3D**.

```
import random
import math

random.seed(1)
n_1d = 20
xs = [random.randint(0, 10_000) for _ in range(n_1d)]
print("1D (datos):", xs)
```

```
1D (datos): [2201, 9325, 1033, 4179, 1931, 8117, 7364, 7737, 6219, 3439, 1537, 7993, 464, 6386, 7090, 9952, 34, 7297, 4363, 3748]
```

1D — Primer intento: Fuerza bruta

Probamos **todas las parejas** y nos quedamos con la distancia mínima.

- Número de parejas: $\binom{n}{2} = \frac{n(n-1)}{2}$.
- Trabajo por pareja constante (resta y valor absoluto).
- **Complejidad temporal:** $O(n^2)$.
- **Complejidad espacial:** $O(1)$ (sin contar el espacio de entrada).

```
def closest_1d_bruteforce(xs):
    best = (xs[0], xs[1])
    best_d = abs(xs[0] - xs[1])

    for i in range(len(xs)):
        for j in range(i + 1, len(xs)):
            d = abs(xs[i] - xs[j])
            if d < best_d:
                best_d = d
                best = (xs[i], xs[j])

    return best, best_d


pair_bf, d_bf = closest_1d_bruteforce(xs)
print("\n1D | Fuerza bruta")
print("  par más cercano:", pair_bf, "| d =", d_bf)
```

```
1D | Fuerza bruta
  par más cercano: (7364, 7297) | d = 67
```

1D — Segundo intento: divide y vencerás (ordenar + vecinos)

Si ordenamos la lista, el par más cercano debe estar entre elementos contiguos. Si dos valores no fueran contiguos, habría otro valor entre ellos que reduciría la distancia.

Pasos:

1. Ordenar n números.
2. Comparar distancias solo entre vecinos consecutivos.

- Ordenación: $O(n \log n)$ (coste dominante).
- Comparación de vecinos: $O(n)$.
- **Complejidad temporal total:** $O(n \log n)$.
- **Complejidad espacial**: $O(n)$ por la ordenación.

```
def closest_1d_dv(xs):
    ys = sorted(xs)                 # paso dominante: O(n log n)
    best = (ys[0], ys[1])
    best_d = ys[1] - ys[0]

    for a, b in zip(ys, ys[1:]):    # O(n)
        d = b - a
        if d < best_d:
            best_d = d
            best = (a, b)

    return best, best_d


pair_dv, d_dv = closest_1d_dv(xs)
print("\n1D | Divide y Vencerás (ordenar + vecinos)")
print("  par más cercano:", pair_dv, "| d =", d_dv)
print("  ¿misma distancia que fuerza bruta?:", d_bf == d_dv)
```

```
1D | Divide y Vencerás (ordenar + vecinos)
  par más cercano: (7297, 7364) | d = 67
  ¿misma distancia que fuerza bruta?: True
```

Extensión a 2D y 3D

La extensión más simple pasa por aplicar **fuerza bruta** a todas las parejas de puntos, calculando la distancia euclídea.

- Distancia en 2D/3D cuesta $O(d)$ con d constante (2 o 3).
- Por tanto, la complejidad temporal sigue siendo $O(n^2)$.

```
def dist2(p, q):
    # distancia euclídea al cuadrado (evita la raíz cuadrada, que es costosa y no afecta a la comparación)
    return sum((a - b) ** 2 for a, b in zip(p, q))

def closest_bruteforce(points):
    best_p, best_q = points[0], points[1]
    best_d2 = dist2(best_p, best_q)

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d2_ = dist2(points[i], points[j])
            if d2_ < best_d2:
                best_d2 = d2_
                best_p, best_q = points[i], points[j]

    return best_p, best_q, math.sqrt(best_d2)


# Extensión a 2D
random.seed(2)
n_2d = 30
pts2 = [(random.randint(0, 10_000), random.randint(0, 10_000)) for _ in range(n_2d)]
p, q, d = closest_bruteforce(pts2)

print("\n2D (muestra):", pts2[:5], "...")
print("2D | Fuerza bruta")
print("  par más cercano:", p, "↔", q, f"| d = {d:.4f}")

# Extensión a 3D
random.seed(3)
n_3d = 40
pts3 = [(random.randint(0, 10_000), random.randint(0, 10_000), random.randint(0, 10_000)) for _ in range(n_3d)]
p, q, d = closest_bruteforce(pts3)

print("\n3D (muestra):", pts3[:5], "...")
print("3D | Fuerza bruta")
print("  par más cercano:", p, "↔", q, f"| d = {d:.4f}")
```

```
2D (muestra): [(926, 1500), (1390, 5915), (2770, 5048), (4121, 9927), (3476, 9941)] ...
2D | Fuerza bruta
```

```
  par más cercano: (6447, 8340) ↔ (6793, 8607) | d = 437.0412

3D (muestra): [(3898, 9709, 8916), (2136, 6061, 9894), (7766, 9516, 1073), (9922, 215, 7687), (4249, 9024, 3839)] ...
3D | Fuerza bruta
  par más cercano: (701, 4935, 508) ↔ (590, 5073, 118) | d = 428.3281
```